

API IndexedDB

Damian Nicolalde-R.

danicolalde@puce.edu.ec

Temas:

- Datos estructurados
 - Base de datos
 - Objetos y almacenes de objetos
 - Índices
 - Transacciones
- Implementación
 - Abriendo la base de datos
 - Definiendo índices
 - Agregando objetos
 - Leyendo objetos
- Listando datos
- Eliminando datos
- Buscando datos

IndexedDB

Una base de datos NoSQL que podemos utilizar para almacenar información localmente en el navegador cuando desarrollamos aplicaciones web.

Es una base de datos clave/valor.

Datos estructurados

- Cuando necesitamos trabajar con grandes cantidades de datos estructurados, debemos recurrir a un sistema de base de datos. Para este propósito, los navegadores incluyen la API IndexedDB.
- La estructura de datos propuesta por la API IndexedDB es diferente de SQL u otros sistemas populares de bases de datos. La información se almacena en la base de datos como objetos (registros) dentro de lo que llamamos *Almacenes de objetos* (tablas). Los Almacenes de objetos no tienen una estructura predefinida, solo un nombre e índices para encontrar los objetos en su interior. Estos objetos tampoco tienen una estructura predefinida; pueden ser diferentes entre ellos y tan complejos como necesitemos. La única condición que deben cumplir los objetos es tener al menos una propiedad declarada como índice para poder encontrarlos dentro de los Almacenes de objetos.

Base de datos

- La base de datos misma es sencilla. Debido a que cada base de datos se asocia a un ordenador y un sitio web o aplicación, no hay usuarios que tengamos que asignar o cualquier otra restricción de acceso que debamos considerar. Solo tenemos que especificar el nombre y la versión, y la base de datos estará lista.
- El objeto **Window** incluye la propiedad **indexedDB** para acceder a la base de datos. Esta propiedad almacena un objeto con los siguientes métodos.
 - **open(nombre, versión)**—Este método crea o abre una base de datos con el nombre y la versión especificados por los atributos. La versión se puede ignorar cuando se crea la base, pero es obligatoria cuando queremos especificar una nueva versión para una base de datos existente. El método devuelve un objeto que desencadena dos eventos (**error** y **success**) para indicar error o éxito en la creación o apertura de la base de datos.
 - **deleteDatabase(nombre)**—Este método elimina la base de datos con el nombre especificado por el atributo.

Objetos y almacenes de objetos

Lo que normalmente llamamos *registros* en una base de datos estándar se llaman *objetos* en la API IndexedDB. Estos objetos incluyen propiedades para almacenar e identificar valores. El número de propiedades y cómo se estructuran los objetos es irrelevante; solo tienen que incluir al menos una propiedad declarada como índice para que el Almacén de objetos pueda encontrarlos.

Del mismo modo, lo que llamamos *tablas* en una base de datos estándar se denomina *Almacenes de objetos* en la API IndexedDB, porque almacenan objetos con la información que queremos preservar. Los Almacenes de objetos tampoco tienen una estructura particular, solo se deben declarar el nombre y uno o más índices cuando se crean los objetos en su interior.

- Para trabajar con objetos y Almacenes de objetos, necesitamos crear el Almacén de objetos, declarar las propiedades que se usarán como índices y luego comenzar a almacenar los objetos en el mismo. Actualmente, no tenemos que pensar en la estructura y el contenido de los objetos, solo tenemos que considerar los índices que vamos a necesitar para realizar búsquedas más adelante. Los siguientes son los métodos que facilita la API para gestionar Almacenes de objetos.
 - **createObjectStore(nombre, objeto)**—Este método crea un nuevo Almacén de objetos con el nombre y la configuración especificados por los atributos (el atributo **nombre** es obligatorio). El atributo **objeto** es un objeto que puede incluir las propiedades **keyPath** y **autoIncrement**. La propiedad **keyPath** declara un índice común para todos los objetos y la propiedad **autoIncrement** es una propiedad booleana que determina si el Almacén de objetos tendrá un generador de claves para organizar los objetos.
 - **deleteObjectStore(nombre)**—Este método elimina el Almacén de objetos con el nombre declarado por el atributo.
 - **objectStore(nombre)**—Este método abre el Almacén de objetos con el nombre declarado por el atributo.

- Los métodos **createObjectStore()** y **deleteObjectStore()**, así como otros métodos responsables de la configuración de la base de datos, solo se pueden aplicar cuando se crea la base de datos.
- La API IndexedDB facilita los siguientes métodos para interactuar con el Almacén de objetos, leer y almacenar información.
 - **add(objeto)**—Este método recibe un par nombre,valor o un objeto que contiene varios pares nombre,valor y agrega un objeto al Almacén de objetos con esta información. Si ya existe un objeto con el mismo índice, este método devuelve un error.
 - **put(objeto)**—Este método es similar al método **add()**, excepto porque sobrescribe un objeto existente con el mismo índice. Es útil para actualizar un objeto que ya se ha almacenado en el Almacén de objetos.
 - **get(nombre)**—Este método recupera un objeto del Almacén de objetos. El atributo **nombre** es el valor del índice del objeto que queremos leer.
 - **delete(nombre)**—Este método elimina un objeto del Almacén de objetos. El atributo **nombre** es el valor del índice del objeto que queremos eliminar.

Índices

- Como ya hemos mencionado, tenemos que declarar algunas de las propiedades de los objetos como índices para encontrarlos luego en el Almacén de objetos. Una manera es declarando la propiedad **keyPath** en el método **createObjectStore()**. La propiedad declarada como **keyPath** será el índice común compartido por todos los objetos almacenados en ese Almacén de objetos en concreto. Cuando declaramos el valor de **keyPath**, la propiedad declarada como índice debe estar presente en todos los objetos.
- Además de **keyPath**, también podemos declarar los índices que queremos para un Almacén de objetos usando los siguientes métodos.
 - **createIndex(nombre, propiedad, objeto)**—Este método crea un índice para un Almacén de objetos. El atributo **nombre** es un nombre que identifica el índice, el atributo **propiedad** es la propiedad del objeto usada como índice y el atributo **objeto** es un objeto que puede incluir las propiedades **unique** o **multiEntry**. La propiedad **unique** indica si el valor del índice debe ser único o hay varios objetos que pueden compartir el mismo valor. La propiedad **multiEntry** determina cuántas claves se generarán para un objeto cuando el índice es un array.
 - **deleteIndex(nombre)**—Este método elimina un índice. Solo se puede llamar desde una transacción de tipo **versionchange**.
 - **index(nombre)**—Este método devuelve una referencia al índice con el nombre especificado por el atributo.

Transacciones

- Un sistema de base de datos que funciona en un navegador debe considerar circunstancias únicas que no están presentes en otras plataformas. Por ejemplo, el navegador puede fallar, se puede cerrar abruptamente, el usuario puede detener el proceso o el usuario puede abandonar nuestro sitio web en medio de una operación. Existen muchas situaciones en las que trabajar directamente con la base de datos puede causar un mal funcionamiento o incluso corromper los datos. Con el fin de prevenir que esto suceda, se debe realizar cada acción a través de transacciones. La API IndexedDB ofrece el siguiente método para realizar una transacción.

- **transaction**(almacén, tipo)—Este método realiza una transacción. El atributo **almacén** es el nombre del Almacén de objetos involucrado en la transacción y el atributo **tipo** es una cadena de caracteres que describe el tipo de transacción que queremos realizar.
- Los siguientes son los tipos de transacciones disponibles para este método.
 - **readonly**—Esta es una transacción de solo lectura. No se permiten modificaciones.
 - **readwrite**—Esta es una transacción de lectura–escritura. Usando este tipo de transacción podemos leer y escribir en la base de datos. Se permiten modificaciones, pero no podemos agregar o eliminar Almacenes de objetos e índices.
 - **versionchange**—Este tipo de transacción se usa para actualizar la versión de la base de datos, así como para agregar o eliminar Almacenes de objetos e índices.

- Las transacciones más comunes son **readwrite**. Sin embargo, para prevenir un uso inadecuado, se declara por defecto el tipo **readonly**, por lo que si solo necesitamos obtener información de la base de datos, solo tenemos que especificar el objetivo de la transacción (generalmente el nombre del Almacén de objetos) y el tipo de transacción se declara automáticamente.
- Se facilitan los siguientes eventos para controlar la transacción.
 - **complete**—Este evento se desencadena cuando se completa la transacción.
 - **abort**—Este evento se desencadena cuando se anula la transacción.
 - **error**—Este evento se desencadena cuando ocurre un error durante la transacción.

1. Implementación

- Es hora de crear nuestra primera base de datos y aplicar algunos de los métodos que hemos mencionado. En esta sección vamos a simular una aplicación para almacenar información sobre películas. Estos son los datos que vamos a utilizar en nuestros ejemplos.
 - **id:** tt0068646 **nombre:** El Padrino **fecha:** 1972
 - **id:** tt0086567 **nombre:** Juegos de Guerra **fecha:** 1983
 - **id:** tt0111161 **nombre:** Cadena Perpetua **fecha:** 1994
 - **id:** tt1285016 **nombre:** La Red Social **fecha:** 2010

```

1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>API IndexedDB</title>
6          <link rel="stylesheet" href="indexed.css">
7          <script src="indexed.js"></script>
8      </head>
9      <body>
10         <section id="cajaformulario">
11             <form name="formulario">
12                 <label for="clave">Clave: </label><br>
13                 <input type="text" name="clave" id="clave"><br>
14                 <label for="texto">Título: </label><br>
15                 <input type="text" name="texto" id="texto"><br>
16                 <label for="fecha">Año: </label><br>
17                 <input type="text" name="fecha" id="fecha"><br>
18                 <button type="button" id="grabar">Grabar</button>
19             </form>
20         </section>
21         <section id="cajadatos">
22             <p>Información no disponible</p>
23         </section>
24     </body>
25 </html>

```

```

1  #cajaformulario {
2      float: left;
3      padding: 20px;
4      border: 1px solid #999999;
5  }
6  #cajadatos {
7      float: left;
8      width: 400px;
9      margin-left: 20px;
10     padding: 20px;
11     border: 1px solid #999999;
12 }
13 #clave, #texto {
14     width: 200px;
15 }
16 #cajadatos > div {
17     padding: 5px;
18     border-bottom: 1px solid #999999;
19 }

```

Clave:

Título:

Año:

Grabar

Información no disponible

Abriendo la base de datos

- El primer paso en el código JavaScript es abrir la base de datos. El método **open()** del objeto **indexedDB** abre la base de datos con el nombre especificado o crea una nueva si no existe.

Indexed.js

```
1  // abriendo la base de datos
2  var cajadatos, bd;
3  function iniciar() {
4      cajadatos = document.getElementById("cajadatos");
5      var boton = document.getElementById("grabar");
6      boton.addEventListener("click", agregarobjeto);
7      var solicitud = indexedDB.open("basededatos");
8      solicitud.addEventListener("error", mostrarerror);
9      solicitud.addEventListener("success", comenzar);
10     solicitud.addEventListener("upgradeneeded", crearbd);
11 }
```

La función **iniciar()** crea las referencias a los elementos del documento y abre la base de datos. El método **open()** intenta abrir una base de datos con el nombre "basededatos" y devuelve un objeto con el resultado. Debido a que la API IndexedDB es una API asíncrona, los eventos **error**, **success** y **upgradeneeded** se desencadenan desde este objeto para informar del resultado de la operación. Cuando se desencadenan los eventos **error** y **success**, se ejecutan las funciones **mostrarerror()** y **comenzar()** para controlar los errores o continuar con la definición de la base de datos.

```
13  ✓ function mostrarerror(evento) {  
14      |     alert("Error: " + evento.code + " " + evento.message);  
15      |  
16      |  
17  ✓ function comenzar(evento) {  
18      |     |     bd = evento.target.result;  
19      |  
--
```

Debido a que en esta aplicación no necesitamos procesar errores, las funciones **mostrarerror()** y **comenzar()** son sencillas. La información del error se muestra usando las propiedades **code** y **message** del objeto **Event**, y se captura una referencia a la base de datos mediante la propiedad **result** y luego se almacena en la variable global **bd**. Esta variable se usará para acceder a la base de datos desde el resto del código.

- Las funciones que responden a estos eventos reciben un objeto de tipo **IDBRequest**. Estos tipos de objetos incluyen propiedades con información acerca de la operación. Las siguientes son las que más se usan.
 - **result**—Esta propiedad devuelve un objeto con el resultado de la solicitud. Este puede ser un objeto que representa la base de datos o un objeto que representa el objeto obtenido del Almacén de objetos.
 - **source**—Esta propiedad devuelve un objeto con información acerca de la fuente de la solicitud.
 - **transaction**—Esta propiedad devuelve un objeto con información acerca de la transacción.
 - **readyState**—Esta propiedad devuelve la condición actual de la transacción. Los valores disponibles son **pending** y **done**.
 - **error**—Esta propiedad devuelve el error de la solicitud.
- En nuestro ejemplo, vamos a usar solo la propiedad **result** para obtener una referencia a la base de datos y los objetos desde los Almacenes de objetos.

Definiendo índices

- En el caso de que se declare una nueva versión de la base de datos mediante el método **open()** o de que no exista la base de datos, el evento **upgradeneeded** se desencadena y se llama a la función **crearbd()** para responder al evento. En este momento, tenemos que pensar en la clase de objetos que necesitamos almacenar en la base de datos y cómo vamos a obtener esta información después desde los Almacenes de objetos.

```
21  ✓ function crearbd(evento) {  
22      var basededatos = evento.target.result;  
23      var almacen = basededatos.createObjectStore("peliculas", {keyPath: "id"});  
24      almacen.createIndex("BuscarFecha", "fecha", {unique: false});  
25  }  
26  }
```

- Para nuestro ejemplo, solo necesitamos un Almacén de objetos (para almacenar las películas) y dos índices. El primer índice, **id**, se declara como el atributo **keyPath** del método **createObjectStore()** cuando se crea el Almacén de objetos, pero el segundo índice se asigna al Almacén de objetos usando el método **createIndex()**. Este índice se identifica con el nombre **BuscarFecha** y se declara para la propiedad **fecha**. Vamos a usar este índice para ordenar las películas por año.
- En esta función hemos tenido que obtener la referencia a la base de datos nuevamente desde la propiedad **result** porque el evento **success** aún no se ha desencadenado y la referencia a la base de datos todavía no se ha creado en nuestro código (el valor de la variable **bd** aún no se ha definido).

Agregando objetos

- Hasta aquí tenemos una base de datos con el nombre *basededatos* y un Almacén de objetos llamado *peliculas* con dos índices: **id** y **fecha** (llamado **BuscarFecha**). Es hora de comenzar a agregar objetos a este almacén.

```

27 //agregar objetos
28 function agregarobjeto() {
29     var clave = document.getElementById("clave").value;
30     var titulo = document.getElementById("texto").value;
31     var fecha = document.getElementById("fecha").value;
32     var transaccion = bd.transaction(["peliculas"], "readwrite");
33     var almacen = transaccion.objectStore("peliculas");
34     transaccion.addEventListener("complete", function() {
35         mostrar(clave);
36     });
37     var solicitud = almacen.add({id: clave, nombre: titulo, fecha: fecha});
38     document.getElementById("clave").value = "";
39     document.getElementById("texto").value = "";
40     document.getElementById("fecha").value = "";
41 }

```

Al comienzo de la función **iniciar()**, agregamos un listener para el evento **click** al botón del formulario. La función **agregarobjeto()** se ejecuta cuando se desencadena este evento. Esta función toma los valores insertados en el formulario (**clave**, **texto** y **fecha**) y genera una transacción para almacenar un nuevo objeto usando esta información.

La transacción se inicia llamando al método **transaction()** y especificando el Almacén de objetos involucrado en la transacción y su tipo. En este caso, el único Almacén disponible es películas y el tipo se declara como **readwrite**.

El siguiente paso es seleccionar el Almacén de objetos que vamos a utilizar. Debido a que la transacción puede iniciar para varios Almacenes de objetos, tenemos que declarar cuál corresponde a la operación que queremos realizar. Con el método **objectStore()** abrimos el Almacén de objetos y lo asignamos a la transacción con la instrucción **transaccion.objectStore("peliculas")**.

La mayoría de las transacciones van a ser consultas a la base de datos y, en esos casos, podemos responder al evento **success** de la solicitud para obtener los resultados (como veremos pronto), pero este evento se puede desencadenar antes de que se produzca un error en la transacción. Por ello, cuando agregamos o modificamos el contenido de la base de datos, es mejor responder al evento **complete** de la transacción. Es por esto que en nuestro ejemplo hemos asignado la función **mostrar()** a este evento. Como siempre, también existe un evento **error**, pero como la respuesta a este evento depende de los requerimientos de la aplicación, no vamos a considerar esta posibilidad en nuestro ejemplo.

Finalmente, es hora de agregar el objeto al Almacén de objetos. En este caso, usamos el método **add()** porque queremos crear nuevos objetos, pero podríamos haber usado el método **put()** si hubiésemos querido modificar o reemplazar objetos existentes. El método **add()** usa las propiedades **id**, **nombre** y **fecha**, y las variables **clave**, **titulo** y **fecha**, y crea el objeto usando estos valores como pares nombrefvalor.

Leyendo objetos

- Si el objeto se almacena correctamente, se desencadena el evento **complete** y se ejecuta la función **mostrar()**. En el código anterior esta función se ha llamado desde dentro de una función anónima para poder pasar la variable **clave**. Ahora vamos a usar este valor para leer el objeto que acabamos de almacenar.

```
43  //leyendo objetos
44  function mostrar(clave) {
45      var transaccion = bd.transaction(["peliculas"]);
46      var almacen = transaccion.objectStore("peliculas");
47      var solicitud = almacen.get(clave);
48      solicitud.addEventListener("success", mostrarlista);
49  }
50  function mostrarlista(evento) {
51      var resultado = evento.target.result;
52      cajadatos.innerHTML = "<div>" + resultado.id + " - " +
53      resultado.nombre + " - " + resultado.fecha + "</div>";
54  }
55
56  window.addEventListener("load", iniciar);
```

Para finalizar la aplicación, debemos responder al evento **load** para ejecutar la función **iniciar()** después de que se cargue el documento.

Las funciones del Listado 15-7 generan una transacción **readonly** y usan el método **get()** para obtener un objeto con el nombre recibido (no tenemos que declarar el tipo de transacción porque el tipo **readonly** se declara por defecto).

El método **get()** devuelve el objeto almacenado con la propiedad **id = clave**. Si, por ejemplo, insertamos la película *El Padrino*, la variable **clave** tendrá el valor "tt0068646". Este valor lo recibe la función **mostrar()** y lo usa el método **get()** para obtener la película *El Padrino*.

Debido a que toda operación es asíncrona, necesitamos dos funciones para mostrar esta información. La función **mostrar()** genera la transacción y la función **mostrarlista()** muestra el valor de las propiedades en la pantalla en caso de éxito.

2. Listando datos

- El método **get()** implementado en el código anterior devuelve solo un objeto a la vez (la última película insertada por el usuario). Para generar una lista que incluya todos los objetos almacenados en un Almacén de objetos, tenemos que usar un cursor.

Cursores

EL cursor es la herramienta que facilita la API para leer y navegar a través de un grupo de objetos que devuelve la base de datos en una transacción. El cursor obtiene una lista de objetos desde el Almacén de objetos e inicializa un puntero que apunta a uno de los objetos de la lista a la vez.

La API incluye el método **openCursor()** para generar un cursor. Este método extrae información desde el Almacén de objetos seleccionado y devuelve un objeto **IDBCursor** que tiene sus propios métodos para gestionar el cursor.

- **continue(índice)**—Este método mueve el puntero del cursor hacia adelante una posición y desencadena el evento **success** en el cursor para informar que la operación ha finalizado. El evento envía los valores del objeto obtenido a la función que responde al mismo. Cuando el puntero alcanza el final de la lista, el evento **success** también se desencadena, pero el objeto enviado a la función es un objeto vacío. El puntero se puede mover a una posición específica declarando un índice como atributo.
- **delete()**—Este método elimina el objeto en la posición del cursor.
- **update(valor)**—Este método es similar a **put()**, pero actualiza el valor del objeto en la posición actual del cursor.

En el siguiente código se indica como trabajar con un cursor.

```

1 var cajadatos, bd;
2 function iniciar() {
3     cajadatos = document.getElementById("cajadatos");
4     var boton = document.getElementById("grabar");
5     boton.addEventListener("click", agregarobjeto);
6     var solicitud = indexedDB.open("basededatos");
7     solicitud.addEventListener("error", mostrarerror);
8     solicitud.addEventListener("success", comenzar);
9     solicitud.addEventListener("upgradeneeded", crearbd);
10 }
11 function mostrarerror(evento) {
12     alert("Error: " + evento.code + " " + evento.message);
13 }
14 function comenzar(evento) {
15     bd = evento.target.result;
16     mostrar();
17 }
18 function crearbd(evento) {
19     var basededatos = evento.target.result;
20     var almacen = basededatos.createObjectStore("peliculas", {keyPath: "id"});
21     almacen.createIndex("BuscarFecha", "fecha", {unique: false});
22 }
23 function agregarobjeto() {
24     var clave = document.getElementById("clave").value;
25     var titulo = document.getElementById("texto").value;
26     var fecha = document.getElementById("fecha").value;
27     var transaccion = bd.transaction(["peliculas"], "readwrite");
28     var almacen = transaccion.objectStore("peliculas");
29     transaccion.addEventListener("complete", mostrar);
30     var solicitud = almacen.add({id: clave, nombre: titulo, fecha: fecha});
31     document.getElementById("clave").value = "";
32     document.getElementById("texto").value = "";
33     document.getElementById("fecha").value = "";
34 }
35 function mostrar() { cajadatos.innerHTML = "";
36     var transaccion = bd.transaction(["peliculas"]);
37     var almacen = transaccion.objectStore("peliculas");
38     var puntero = almacen.openCursor();
39     puntero.addEventListener("success", mostrarlista);
40 }
41 function mostrarlista(evento) {
42     var puntero = evento.target.result;
43     if (puntero) {
44         cajadatos.innerHTML += "<div> " + puntero.value.id + " - " + puntero.value.nombre + " - " + puntero.value.fecha + "</div>";
45         puntero.continue();
46     }
47 }
48 window.addEventListener("load", iniciar);

```

De todas las funciones utilizadas para inicializar y configurar la base de datos, solo **comenzar()** presenta un cambio. Ahora, la función **mostrar()** se llama al final de esta función para mostrar la lista de objetos dentro del Almacén de objetos en la pantalla tan pronto como se carga el documento. Los cambios significativos en este código se encuentran en las funciones **mostrar()** y **mostrarlista()**, donde trabajamos con cursores por primera vez.

La lectura de información de la base de datos con un cursor es también una operación que debe realizarse a través de una transacción. En consecuencia, el primer paso en la función **mostrar()** es generar una transacción **readonly** para el Almacén de objetos películas. Este Almacén de objetos se selecciona para esta transacción y luego el cursor se abre con el método **openCursor()**. Si la operación se realiza correctamente, se devuelve un objeto con la información obtenida del Almacén de objetos, un evento **success** se desencadena desde este objeto y se ejecuta la función **mostrarlista()**.

En la función **mostrarlista()** usamos una instrucción **if** para controlar el contenido del cursor. Si no se devuelve ningún objeto, o el puntero alcanza el final de la lista, el valor de la variable cursor será null y el bucle finalizará. Sin embargo, cuando el puntero apunta a un objeto válido, la información se muestra en pantalla y el puntero se mueve a la siguiente posición con **continue()**.

Es importante mencionar que no tenemos que usar un bucle en este caso porque el método **continue()** desencadena el evento **success** y toda la función se ejecuta otra vez hasta que el cursor devuelve null.

➔ Indexed1.js

El objeto recibido por la función **mostrarlista()** incluye las siguientes propiedades para leer la información.

- **key**—Esta propiedad devuelve el nombre del objeto en la posición actual del cursor. **value**—Esta propiedad devuelve un objeto con los valores de las propiedades del objeto en la posición actual del cursor.
- **direction**—Esta propiedad devuelve el orden en el que se leen objetos (ascendente o descendente).
- **count**—Esta propiedad devuelve el número aproximado de objetos en el cursor.

Clave:	
Título:	
Año:	
Grabar	

--
12 - we - 34
2 - b - 1990
3 - c - 2000
4 - er - 1900
P001 - Duro de Matar - 2008
P002 - El principe del rap - 1994
P003 - El hombre araña - 1998

Orden

- Las películas de nuestro ejemplo se listan en orden ascendente y la propiedad que se usa para organizar los objetos es **id**. Esta propiedad es el valor de **keyPath** para el Almacén de objetos películas, pero no es un valor que será de interés para los usuarios. Considerando esta situación, en la función **crearbd()** agregamos otro índice a nuestro almacén. El nombre de este índice adicional es **BuscarFecha** y la propiedad asignada al índice es **fecha**. Este índice nos permite ordenar las películas según el año en que se filmaron. Los siguientes son los cambios que necesitamos introducir en la función **mostrar()** con este propósito.

```

35 function mostrar() {
36     cajadatos.innerHTML = "";
37     var transaccion = bd.transaction(["peliculas"]);
38     var almacen = transaccion.objectStore("peliculas");
39     var indice = almacen.index("BuscarFecha");
40     var puntero = indice.openCursor(null, "prev");
41     puntero.addEventListener("success", mostrarlista);
42 }

```

Indexed2.js

Clave:
Título:
Año:

12 - we - 34
P005 - Batman - 2020
P001 - Duro de Matar - 2008
3 - c - 2000
P003 - El hombre araña - 1998
P002 - El principe del rap - 1994
2 - b - 1990
P004 - Superman - 1905
4 - er - 1900
- -

La función **mostrar()** tiene un cambio. Esta nueva función genera una transacción, luego asigna el índice **BuscarFecha** al Almacén de objetos usado en la transacción y finalmente llama al método **openCursor()** para obtener la lista de objetos que contienen la propiedad correspondiente a ese índice (en este caso, **fecha**).

Existen dos atributos que podemos incluir en el método **openCursor()** para seleccionar y ordenar la información que devuelve el cursor. El primer atributo especifica el rango con el que seleccionar los objetos y el segundo atributo es una de las siguientes constantes.

- **next**—Los objetos se devuelven en orden ascendente (por defecto).
- **nextunique**—Los objetos se devuelven en orden ascendente y los objetos duplicados se ignoran (solo se devuelve el primer objeto si se encuentra un nombre duplicado).
- **prev**—Los objetos se devuelven en orden ascendente.
- **prevunique**—Los objetos se devuelven en orden descendente y los objetos duplicados se ignoran (solo se devuelve el primer objeto si se encuentra un nombre duplicado).

En el método **openCursor()** dentro de la función **mostrar()** declaramos el atributo para el rango como **null** y el orden de los objetos como descendiente. Estos valores devuelven todos los objetos en orden descendiente.

3. Eliminando datos

- Ya hemos aprendido a agregar, leer y listar datos. Es hora de ver cómo eliminar objetos del Almacén de objetos. Como hemos mencionado antes, el método **delete()** que facilita la API recibe un valor y elimina el objeto con el nombre correspondiente a ese valor. El código para nuestro ejemplo es sencillo; solo tenemos que modificar la función **mostrarlista()** para crear botones por cada objeto listado en la pantalla y generar una transacción **readwrite** para poder eliminarlos cuando se eliminan los botones.

```

44 function mostrarlista(evento) {
45     var puntero = evento.target.result;
46     if (puntero) {
47         cajadatos.innerHTML += "<div>" + puntero.value.id + " - " + puntero.value.nombre + " - ";
48         cajadatos.innerHTML += puntero.value.fecha + ' <input type="button" onclick="removerobjeto(\'' + puntero.value.id + '\')" value="Remover"></div>';
49         puntero.continue();
50     }
51 }
52
53 function removerobjeto(clave) {
54     if (confirm("Está seguro?")) {
55         var transaccion = bd.transaction(["peliculas"], "readwrite");
56         var almacen = transaccion.objectStore("peliculas");
57         transaccion.addEventListener("complete", mostrar);
58         var solicitud = almacen.delete(clave);
59     }
60 }

```

Los botones agregados a cada objeto en la función **mostrarlista()** contienen un atributo de evento. Cada vez que el usuario hace clic en uno de estos botones, la función **removerobjeto()** se ejecuta con el valor de la propiedad **id** del objeto como atributo. Esta función genera una transacción **readwrite** y luego, usando el valor del atributo **clave**, procede a eliminar el objeto correspondiente del Almacén de objetos películas. Al final, si la transacción se realiza correctamente, se desencadena el evento **complete** y se ejecuta la función **mostrar()** para actualizar la lista de películas en pantalla.

Clave:

Título:

Año:

Grabar

P005 - Batman -	
2020	<button>Remover</button>
P006 - Lost -	
2015	<button>Remover</button>
P001 - Duro de Matar -	
2008	<button>Remover</button>
P007 - Juego Mortal -	
1998	<button>Remover</button>
P003 - El hombre araña -	
1998	<button>Remover</button>

4. Buscando datos

- La operación más importante que se realiza en una base de datos es probablemente la búsqueda de datos. El propósito de este tipo de sistemas es el de organizar la información para que sea fácil de encontrar. El método **get()** es útil para obtener un objeto a la vez cuando conocemos su nombre, pero una operación de búsqueda es más complicada.
- Con el fin de obtener una lista específica de objetos desde el Almacén de objetos, tenemos que especificar un rango como el primer atributo del método **openCursor()**. La API facilita el objeto **IDBKeyRange** con varios métodos para declarar un rango y filtrar los objetos devueltos.
 - **only(valor)**—Este método devuelve un rango donde solo se devuelven los objetos con el valor igual al atributo **valor**. Por ejemplo, si buscamos películas por año usando **only("2020")**.
 - **bound(bajo, alto, bajoAbierto, altoAbierto)**—Este método devuelve un rango cerrado con valores de inicio y finalización. El atributo **bajo** especifica el valor de inicio del rango, el atributo **alto** especifica el valor final del rango y los atributos **bajoAbierto** y **altoAbierto** son valores booleanos que declaran si se ignorarán los objetos que coinciden con los valores de los atributos **bajo** y **alto**. Por ejemplo, **bound("1972", "2010", false, true)** devuelve la lista de películas filmadas desde 1972 a 2010, pero no incluye las realizadas en el año 2010.
 - **lowerBound(valor, abierto)**—Este método crea un rango abierto que comienza por el valor especificado por el atributo **valor** y va hasta el final de la lista. Por ejemplo, **lowerBound("1983", true)** devuelve todas las películas hechas después de 1983, sin incluir las que se filmaron ese año.
 - **upperBound(valor, abierto)**—Este método crea un rango abierto, pero a diferencia del método **lowerBound()** los objetos que devuelve son los que se encuentran desde el inicio de la lista hasta el valor especificado por el atributo **valor**. Por ejemplo, **upperBound("1983", false)** devuelve las películas hechas antes de 1983, incluidas las realizadas ese año.

```
1  <!DOCTYPE html>
2  <html lang="es">
3    <head>
4      <meta charset="utf-8">
5      <title>API IndexedDB</title>
6      <link rel="stylesheet" href="indexed.css">
7      <script src="busqueda.js"></script>
8    </head>
9    <body>
10     <section id="cajaformulario">
11       <form name="formulario">
12         <label for="fecha">Find Movie by Year: </label><br>
13         <input type="text" name="fecha" id="fecha"><br>
14         <button type="button" id="buscar">Buscar</button>
15       </form>
16     </section>
17     <section id="cajadatos">
18       <p>Información no disponible</p>
19     </section>
20   </body>
21 </html>
```

Find Movie by Year:

Buscar

Información no disponible

```

1 var cajadatos, bd;
2 function iniciar() {
3     cajadatos = document.getElementById("cajadatos");
4     var boton = document.getElementById("buscar");
5     boton.addEventListener("click", buscarobjetos);
6     var solicitud = indexedDB.open("basededatos");
7     solicitud.addEventListener("error", mostrarerror);
8     solicitud.addEventListener("success", comenzar);
9     solicitud.addEventListener("upgradeneeded", crearbd);
10 }
11 function mostrarerror(evento) {
12     alert("Error: " + evento.code + " " + evento.message);
13 }
14 function comenzar(evento) {
15     bd = evento.target.result;
16 }
17 function crearbd(evento) {
18     var basededatos = evento.target.result;
19     var almacen = basededatos.createObjectStore("peliculas", {keyPath: "id"});
20     almacen.createIndex("BuscarFecha", "fecha", {unique: false});
21 }
22 function buscarobjetos() {
23     cajadatos.innerHTML = "";
24     var buscar = document.getElementById("fecha").value;
25     var transaccion = bd.transaction(["peliculas"]);
26     var almacen = transaccion.objectStore("peliculas");
27     var indice = almacen.index("BuscarFecha");
28     var rango = IDBKeyRange.only(buscar);
29     var puntero = indice.openCursor(rango); puntero.addEventListener("success", mostrarlista);
30 }
31 function mostrarlista(evento) {
32     var puntero = evento.target.result;
33     if (puntero) {
34         cajadatos.innerHTML += "<div>" + puntero.value.id + " - " + puntero.value.nombre + " - " + puntero.value.fecha + "</div>";
35         puntero.continue();
36     }
37 }
38 window.addEventListener("load", iniciar);

```

Find Movie by Year:

P003 - El hombre araña - 1998

P007 - Juego Mortal - 1998

La función **buscarobjetos()** genera una transacción **readonly** para el Almacén de objetos películas, selecciona el índice **BuscarFecha** para usar la propiedad **fecha** como índice y crea un rango que solo incluye los objetos con un valor igual al valor de la variable **buscar** (el año insertado en el formulario). Este rango se pasa como el atributo del método **openCursor()**. Después de una operación realizada correctamente, la función **mostrarlista()** imprime la lista de películas que coinciden con el año seleccionado en pantalla.